

02244 Logic for Security Information Flow Week 9: Volpano's Approach

Sebastian Mödersheim

April 13, 2026

Challenges for Denning's

We add arrays and procedures to our language

- $D ::= \dots \mid \textbf{integer array } C \ A[n] \mid \dots$
 $\dots \mid \textbf{proc } p(\textbf{in } T_1 \ C_1 \ \text{var}_{\text{in}}, \textbf{out } T_2 \ C_2 \ \text{var}_{\text{out}}) \textbf{ is } S_{\text{body}}$
- $E ::= \dots \mid A[E_1]$
- $S ::= \dots \mid A[E_1] := E_2 \mid \textbf{call } p(E, \text{var})$

Define the new constraints that we need to impose.

Challenges for Denning's

We add arrays and procedures to our language

- $D ::= \dots \mid \mathbf{integer\ array}\ C\ A[n] \mid \dots$
 $\dots \mid \mathbf{proc}\ p(\mathbf{in}\ T_1\ C_1\ \mathbf{var}_{in}, \mathbf{out}\ T_2\ C_2\ \mathbf{var}_{out})\ \mathbf{is}\ S_{body}$
- $E ::= \dots \mid A[E_1]$
- $S ::= \dots \mid A[E_1] := E_2 \mid \mathbf{call}\ p(E, \mathbf{var})$

Rules

D	Security Class
$\mathbf{integer}\ C\ A[n]$	$\underline{A} =$
$\mathbf{proc}\ p(\mathbf{in}\ T_1\ C_1\ \mathbf{var}_{in}, \mathbf{out}\ T_2\ C_2\ \mathbf{var}_{out})\ \mathbf{is}\ S_{body}$	$\underline{p} =$ $\underline{\mathbf{var}_{in}} =$ $\underline{\mathbf{var}_{out}} =$
E	Security Class
$A[E_1]$	$\underline{E} =$

Challenges for Denning's

We add arrays and procedures to our language

- $D ::= \dots \mid \text{integer array } C \ A[n] \mid \dots$
 $\dots \mid \text{proc } p(\text{ in } T_1 \ C_1 \ \text{var}_{\text{in}}, \text{ out } T_2 \ C_2 \ \text{var}_{\text{out}}) \text{ is } S_{\text{body}}$
- $E ::= \dots \mid A[E_1]$
- $S ::= \dots \mid A[E_1] := E_2 \mid \text{call } p(E, \text{var})$

Rules

D	Security Class
integer $C \ A[n]$	$\underline{A} = C$
proc $p(\text{ in } T_1 \ C_1 \ \text{var}_{\text{in}},$ $\text{ out } T_2 \ C_2 \ \text{var}_{\text{out}}) \text{ is } S_{\text{body}}$	$\underline{p} =$ $\underline{\text{var}_{\text{in}}} =$ $\underline{\text{var}_{\text{out}}} =$
E	Security Class
$A[E_1]$	$\underline{E} =$

Challenges for Denning's

We add arrays and procedures to our language

- $D ::= \dots \mid \text{integer array } C \ A[n] \mid \dots$
 $\dots \mid \text{proc } p(\text{ in } T_1 \ C_1 \ \text{var}_{\text{in}}, \text{ out } T_2 \ C_2 \ \text{var}_{\text{out}}) \text{ is } S_{\text{body}}$
- $E ::= \dots \mid A[E_1]$
- $S ::= \dots \mid A[E_1] := E_2 \mid \text{call } p(E, \text{var})$

Rules

D	Security Class
integer $C \ A[n]$	$\underline{A} = C$
proc $p(\text{ in } T_1 \ C_1 \ \text{var}_{\text{in}},$ $\text{ out } T_2 \ C_2 \ \text{var}_{\text{out}}) \text{ is } S_{\text{body}}$	$\underline{p} = \underline{S_{\text{body}}}$ $\underline{\text{var}_{\text{in}}} =$ $\underline{\text{var}_{\text{out}}} =$
E	Security Class
$A[E_1]$	$\underline{E} =$

Challenges for Denning's

We add arrays and procedures to our language

- $D ::= \dots \mid \text{integer array } C \ A[n] \mid \dots$
 $\dots \mid \text{proc } p(\text{ in } T_1 \ C_1 \ \text{var}_{\text{in}}, \text{ out } T_2 \ C_2 \ \text{var}_{\text{out}}) \text{ is } S_{\text{body}}$
- $E ::= \dots \mid A[E_1]$
- $S ::= \dots \mid A[E_1] := E_2 \mid \text{call } p(E, \text{var})$

Rules

D	Security Class
integer $C \ A[n]$	$\underline{A} = C$
proc $p(\text{ in } T_1 \ C_1 \ \text{var}_{\text{in}}, \text{ out } T_2 \ C_2 \ \text{var}_{\text{out}}) \text{ is } S_{\text{body}}$	$\underline{p} = \underline{S_{\text{body}}}$ $\underline{\text{var}_{\text{in}}} = C_1$ $\underline{\text{var}_{\text{out}}} = C_2$
E	Security Class
$A[E_1]$	$\underline{E} =$

Challenges for Denning's

We add arrays and procedures to our language

- $D ::= \dots \mid \text{integer array } C \ A[n] \mid \dots$
 $\dots \mid \text{proc } p(\text{ in } T_1 \ C_1 \ \text{var}_{\text{in}}, \text{ out } T_2 \ C_2 \ \text{var}_{\text{out}}) \text{ is } S_{\text{body}}$
- $E ::= \dots \mid A[E_1]$
- $S ::= \dots \mid A[E_1] := E_2 \mid \text{call } p(E, \text{var})$

Rules

D	Security Class
integer $C \ A[n]$	$\underline{A} = C$
proc $p(\text{ in } T_1 \ C_1 \ \text{var}_{\text{in}}, \text{ out } T_2 \ C_2 \ \text{var}_{\text{out}}) \text{ is } S_{\text{body}}$	$\underline{p} = \underline{S_{\text{body}}}$ $\underline{\text{var}_{\text{in}}} = C_1$ $\underline{\text{var}_{\text{out}}} = C_2$
E	Security Class
$A[E_1]$	$\underline{E} = \underline{A} \sqcup \underline{E_1}$

Challenges for Denning's

We add arrays and procedures to our language

- $D ::= \dots \mid \text{integer array } C \ A[n] \mid \dots$
 $\dots \mid \text{proc } p(\text{in } T_1 \ C_1 \ \text{var}_{in}, \text{out } T_2 \ C_2 \ \text{var}_{out}) \text{ is } S_{\text{body}}$
- $E ::= \dots \mid A[E_1]$
- $S ::= \dots \mid A[E_1] := E_2 \mid \text{call } p(E, \text{var})$

Rules

S	security class of S	constraint
$A[E_1] := E_2$	$\underline{S} =$	
$\text{call } p(E, \text{var})$	$\underline{S} =$	

where p was declared with as input var_{in} and output var_{out} .

Challenges for Denning's

We add arrays and procedures to our language

- $D ::= \dots \mid \text{integer array } C \ A[n] \mid \dots$
 $\dots \mid \text{proc } p(\text{in } T_1 \ C_1 \ \text{var}_{in}, \text{out } T_2 \ C_2 \ \text{var}_{out}) \text{ is } S_{\text{body}}$
- $E ::= \dots \mid A[E_1]$
- $S ::= \dots \mid A[E_1] := E_2 \mid \text{call } p(E, \text{var})$

Rules

S	security class of S	constraint
$A[E_1] := E_2$	$\underline{S} =$	$\underline{E_1} \sqcup \underline{E_2} \sqsubseteq \underline{A}$
$\text{call } p(E, \text{var})$	$\underline{S} =$	

where p was declared with as input var_{in} and output var_{out} .

Challenges for Denning's

We add arrays and procedures to our language

- $D ::= \dots \mid \text{integer array } C \ A[n] \mid \dots$
 $\dots \mid \text{proc } p(\text{in } T_1 \ C_1 \ \text{var}_{in}, \text{out } T_2 \ C_2 \ \text{var}_{out}) \text{ is } S_{body}$
- $E ::= \dots \mid A[E_1]$
- $S ::= \dots \mid A[E_1] := E_2 \mid \text{call } p(E, \text{var})$

Rules

S	security class of S	constraint
$A[E_1] := E_2$	$\underline{S} = \underline{A}$	$\underline{E_1} \sqcup \underline{E_2} \sqsubseteq \underline{A}$
$\text{call } p(E, \text{var})$	$\underline{S} =$	

where p was declared with as input var_{in} and output var_{out} .

Challenges for Denning's

We add arrays and procedures to our language

- $D ::= \dots \mid \text{integer array } C \ A[n] \mid \dots$
 $\dots \mid \text{proc } p(\text{in } T_1 \ C_1 \ \text{var}_{in}, \text{out } T_2 \ C_2 \ \text{var}_{out}) \text{ is } S_{\text{body}}$
- $E ::= \dots \mid A[E_1]$
- $S ::= \dots \mid A[E_1] := E_2 \mid \text{call } p(E, \text{var})$

Rules

S	security class of S	constraint
$A[E_1] := E_2$	$\underline{S} = \underline{A}$	$\underline{E_1} \sqcup \underline{E_2} \sqsubseteq \underline{A}$
$\text{call } p(E, \text{var})$	$\underline{S} = \underline{p}$	

where p was declared with as input var_{in} and output var_{out} .

Challenges for Denning's

We add arrays and procedures to our language

- $D ::= \dots \mid \text{integer array } C \ A[n] \mid \dots$
 $\dots \mid \text{proc } p(\text{in } T_1 \ C_1 \ \text{var}_{in}, \text{out } T_2 \ C_2 \ \text{var}_{out}) \text{ is } S_{\text{body}}$
- $E ::= \dots \mid A[E_1]$
- $S ::= \dots \mid A[E_1] := E_2 \mid \text{call } p(E, \text{var})$

Rules

S	security class of S	constraint
$A[E_1] := E_2$	$\underline{S} = \underline{A}$	$\underline{E_1} \sqcup \underline{E_2} \sqsubseteq \underline{A}$
$\text{call } p(E, \text{var})$	$\underline{S} = \underline{p}$	$\underline{E} \sqsubseteq \underline{\text{var}_{in}}$

where p was declared with as input var_{in} and output var_{out} .

Challenges for Denning's

We add arrays and procedures to our language

- $D ::= \dots \mid \text{integer array } C \ A[n] \mid \dots$
 $\dots \mid \text{proc } p(\text{in } T_1 \ C_1 \ \text{var}_{in}, \text{out } T_2 \ C_2 \ \text{var}_{out}) \text{ is } S_{\text{body}}$
- $E ::= \dots \mid A[E_1]$
- $S ::= \dots \mid A[E_1] := E_2 \mid \text{call } p(E, \text{var})$

Rules

S	security class of S	constraint
$A[E_1] := E_2$	$\underline{S} = \underline{A}$	$\underline{E_1} \sqcup \underline{E_2} \sqsubseteq \underline{A}$
$\text{call } p(E, \text{var})$	$\underline{S} = \underline{p}$	$\underline{E} \sqsubseteq \underline{\text{var}_{in}}$ $\underline{\text{var}_{out}} \sqsubseteq \underline{\text{var}}$

where p was declared with as input var_{in} and output var_{out} .

So What?

Up to this point, we (and Denning and Denning) have only...

- defined a set of security classes
- decorated variables with these security classes
- decorated programs with constraints on security classes

So?

- If a program satisfies the constraints, then what **guarantees** do we really obtain from that?
- Given a set of rules for information flow (like the ones we just gave for arrays and procedures):
how can one **judge** if they are “correct” or “wrong”?

Volpano's Approach to IFC

Dennis M. Volpano and Geoffrey Smith and Cynthia E. Irvine: *A Sound Type System for Secure Flow Analysis*, Journal of Computer Security, 4(2/3), pp. 167–188, 1996. [2]

- Essentially the same approach as Denning and Denning [1]
- ... but represented as a **type system**
 - ★ Security classes like Low and High are considered as **types**
 - ★ The ordering of security classes $\sqsubseteq$ is considered as a **subtype** relation
 - ★ Information Flow Analysis is described as a set of **type-inference rules**
- They give a natural **semantics** for the programming language.
- Type system and semantics allows for proving a precise statement about the security of programs that fulfill the information flow policy: a **Non-Interference Result**.

Volpano's Approach to IFC

Dennis M. Volpano and Geoffrey Smith and Cynthia E. Irvine: *A Sound Type System for Secure Flow Analysis*, Journal of Computer Security, 4(2/3), pp. 167–188, 1996. [2]

We simplify the paper a bit:

- Volpano et al. distinguish *variables* and *memory locations*, we treat them here all as variables (dropping the concept of introducing local variables).
- The corresponding symbol tables γ and λ are merged to just γ .
- We directly work with the syntax-directed form and do not need to distinguish the syntactic roles of variables, expressions and commands in the type system (i.e., we do not have *var* τ etc.)

Syntax

Syntax

Expressions $e ::= x \mid n \mid e + e' \mid \dots \mid e = e' \mid \dots$

Commands $c ::= x := e \mid c; c' \mid \text{if } e \text{ then } c \text{ else } c' \mid \text{while } e \text{ do } c$

where x is for identifiers (for variables) and n is for constants.

Typing Rules

We define **type inference rules** where:

- γ is a **type environment**: it gives for every variable its type.
- The **types** are the security classes SC (e.g. High and Low) and $\sqsubseteq$ is the comparison relation on the security classes.
- **Type judgements** for expressions are of the form

$$\gamma \vdash e : \tau$$

Read: under γ , the expression e **can** be typed as τ .

Typing Rules (Expressions)

$$(\text{CONST}) \quad \overline{\gamma \vdash n : \tau}$$

$$(\text{VAR}) \quad \overline{\gamma \vdash x : \tau} \quad \gamma(x) \sqsubseteq \tau$$

$$(\text{ARITH}) \quad \frac{\gamma \vdash e : \tau \quad \gamma \vdash e' : \tau}{\gamma \vdash e \text{ op } e' : \tau}$$

Typing Rules

- **Type judgements** for commands are of the form

$$\gamma \vdash c : \tau$$

Read: under γ , the command c **can** be typed as τ .

Typing Rules (Commands)

$$\text{(ASSIGN)} \quad \frac{\gamma \vdash e : \tau}{\gamma \vdash x := e : \tau'} \quad \gamma(x) = \tau, \tau' \sqsubseteq \tau$$

$$\text{(COMPOSE)} \quad \frac{\gamma \vdash c : \tau \quad \gamma \vdash c' : \tau}{\gamma \vdash c; c' : \tau}$$

$$\text{(IF)} \quad \frac{\gamma \vdash e : \tau \quad \gamma \vdash c : \tau \quad \gamma \vdash c' : \tau}{\gamma \vdash \text{if } e \text{ then } c \text{ else } c' : \tau'} \quad \tau' \sqsubseteq \tau$$

$$\text{(WHILE)} \quad \frac{\gamma \vdash e : \tau \quad \gamma \vdash c : \tau}{\gamma \vdash \text{while } e \text{ do } c : \tau'} \quad \tau' \sqsubseteq \tau$$

Example

- Let $\gamma(x) = H$ and $\gamma(y) = L$

$$\frac{\text{TODO}}{\gamma \vdash \text{if } x > 1000 \text{ then } x := 5 \text{ else } y := y + x * 3 : \tau_1}$$

Constraints:

Fitting rule

$$(IF) \quad \frac{\gamma \vdash e : \tau \quad \gamma \vdash c : \tau \quad \gamma \vdash c' : \tau}{\gamma \vdash \text{if } e \text{ then } c \text{ else } c' : \tau'} \tau' \sqsubseteq \tau$$

Example

- Let $\gamma(x) = H$ and $\gamma(y) = L$

$$\frac{\frac{TODO}{\gamma \vdash x > 1000 : \tau_2} \quad \frac{TODO}{\gamma \vdash x := 5 : \tau_2} \quad \frac{TODO}{\gamma \vdash y := y + x * 3 : \tau_2}}{\gamma \vdash \text{if } x > 1000 \text{ then } x := 5 \text{ else } y := y + x * 3 : \tau_1}$$

Constraints: $\tau_1 \sqsubseteq \tau_2$

Fitting rule

$$\text{(ARITH)} \quad \frac{\gamma \vdash e : \tau \quad \gamma \vdash e' : \tau}{\gamma \vdash e \text{ op } e' : \tau}$$

Example

- Let $\gamma(x) = H$ and $\gamma(y) = L$

$$\frac{\frac{TODO}{\gamma \vdash x : \tau_2} \quad \frac{TODO}{\gamma \vdash 1000 : \tau_2}}{\gamma \vdash x > 1000 : \tau_2} \quad \frac{TODO}{\gamma \vdash x := 5 : \tau_2} \quad \frac{TODO}{\gamma \vdash y := y + x * 3 : \tau_2}$$
$$\gamma \vdash \text{if } x > 1000 \text{ then } x := 5 \text{ else } y := y + x * 3 : \tau_1$$

Constraints: $\tau_1 \sqsubseteq \tau_2$

Fitting rule

$$(\text{VAR}) \quad \frac{}{\gamma \vdash x : \tau} \quad \gamma(x) \sqsubseteq \tau$$

Example

- Let $\gamma(x) = H$ and $\gamma(y) = L$

$$\frac{\frac{\gamma \vdash x : H}{\gamma \vdash x > 1000 : H} \quad \frac{TODO}{\gamma \vdash 1000 : H}}{\gamma \vdash \text{if } x > 1000 \text{ then } x := 5 \text{ else } y := y + x * 3 : \tau_1} \quad \frac{TODO}{\gamma \vdash x := 5 : H} \quad \frac{TODO}{\gamma \vdash y := y + x * 3 : H}$$

Constraints: $\tau_1 \sqsubseteq \tau_2 \wedge \gamma(x) = H \sqsubseteq \tau_2$ Thus: $\tau_2 = H$.

Fitting rule

(CONST) $\overline{\gamma \vdash n : \tau}$

Example

- Let $\gamma(x) = H$ and $\gamma(y) = L$

$$\frac{\overline{\gamma \vdash x : H} \quad \overline{\gamma \vdash 1000 : H}}{\gamma \vdash x > 1000 : H} \quad \frac{TODO}{\gamma \vdash x := 5 : H} \quad \frac{TODO}{\gamma \vdash y := y + x * 3 : H}$$

$$\gamma \vdash \text{if } x > 1000 \text{ then } x := 5 \text{ else } y := y + x * 3 : \tau_1$$

Constraints:

Fitting rule

$$\text{(ASSIGN)} \quad \frac{\gamma \vdash e : \tau}{\gamma \vdash x := e : \tau'} \quad \gamma(x) = \tau, \tau' \sqsubseteq \tau$$

Example

- Let $\gamma(x) = H$ and $\gamma(y) = L$

$$\frac{\overline{\gamma \vdash x : H} \quad \overline{\gamma \vdash 1000 : H}}{\gamma \vdash x > 1000 : H} \quad \frac{\overline{\gamma \vdash 5 : H}}{\gamma \vdash x := 5 : H} \quad \frac{TODO}{\gamma \vdash y := y + x * 3 : H}$$
$$\gamma \vdash \text{if } x > 1000 \text{ then } x := 5 \text{ else } y := y + x * 3 : \tau_1$$

Constraints:

Fitting rule

$$\text{(ASSIGN)} \quad \frac{\gamma \vdash e : \tau}{\gamma \vdash x := e : \tau'} \quad \gamma(x) = \tau, \tau' \sqsubseteq \tau$$

Example

- Let $\gamma(x) = H$ and $\gamma(y) = L$

$$\frac{\overline{\gamma \vdash x : H} \quad \overline{\gamma \vdash 1000 : H}}{\gamma \vdash x > 1000 : H} \quad \frac{\overline{\gamma \vdash 5 : H}}{\gamma \vdash x := 5 : H} \quad \frac{\text{requires } \gamma(y) \sqsupseteq H!}{\gamma \vdash y := y + x * 3 : H}$$
$$\frac{}{\gamma \vdash \text{if } x > 1000 \text{ then } x := 5 \text{ else } y := y + x * 3 : \tau_1}$$

Constraints:

Fitting rule

$$\text{(ASSIGN)} \quad \frac{\gamma \vdash e : \tau}{\gamma \vdash x := e : \tau'} \quad \gamma(x) = \tau, \tau' \sqsubseteq \tau$$

There is no way to finish the red part of the proof. It does not type!

Example

- Let $\gamma(x) = L$ and $\gamma(y) = H$

$$\frac{\text{TODO}}{\gamma \vdash \text{if } x > 1000 \text{ then } x := 5 \text{ else } y := y + x * 3 : \tau_1}$$

Constraints:

Fitting Rule

$$\text{(IF)} \quad \frac{\gamma \vdash e : \tau \quad \gamma \vdash c : \tau \quad \gamma \vdash c' : \tau}{\gamma \vdash \text{if } e \text{ then } c \text{ else } c' : \tau'} \quad \tau' \sqsubseteq \tau$$

Example

- Let $\gamma(x) = L$ and $\gamma(y) = H$

$$\frac{\frac{TODO}{\gamma \vdash x > 1000 : \tau_2} \quad \frac{TODO}{\gamma \vdash x := 5 : \tau_2} \quad \frac{TODO}{\gamma \vdash y := y + x * 3 : \tau_2}}{\gamma \vdash \text{if } x > 1000 \text{ then } x := 5 \text{ else } y := y + x * 3 : \tau_1}$$

Constraints: $\tau_1 \sqsubseteq \tau_2$

Fitting Rule

$$\text{(ARITH)} \quad \frac{\gamma \vdash e : \tau \quad \gamma \vdash e' : \tau}{\gamma \vdash e \text{ op } e' : \tau}$$

Example

- Let $\gamma(x) = L$ and $\gamma(y) = H$

$$\frac{\frac{\text{TODO}}{\gamma \vdash x : \tau_2} \quad \frac{}{\gamma \vdash 1000 : \tau_2}}{\gamma \vdash x > 1000 : \tau_2} \quad \frac{\text{TODO}}{\gamma \vdash x := 5 : \tau_2} \quad \frac{\text{TODO}}{\gamma \vdash y := y + x * 3 : \tau_2}}{\gamma \vdash \text{if } x > 1000 \text{ then } x := 5 \text{ else } y := y + x * 3 : \tau_1}$$

Constraints: $\tau_1 \sqsubseteq \tau_2$

Fitting Rule

$$(\text{VAR}) \quad \frac{}{\gamma \vdash x : \tau} \quad \gamma(x) \sqsubseteq \tau$$

Example

- Let $\gamma(x) = L$ and $\gamma(y) = H$

$$\frac{\frac{\overline{\gamma \vdash x : \tau_2} \quad \overline{\gamma \vdash 1000 : \tau_2}}{\gamma \vdash x > 1000 : \tau_2} \quad \frac{TODO}{\gamma \vdash x := 5 : \tau_2} \quad \frac{TODO}{\gamma \vdash y := y + x * 3 : \tau_2}}{\gamma \vdash \text{if } x > 1000 \text{ then } x := 5 \text{ else } y := y + x * 3 : \tau_1}$$

Constraints: $\tau_1 \sqsubseteq \tau_2$ $L \sqsubseteq \tau_2$ (trivially true)

Fitting Rule

$$\text{(ASSIGN)} \quad \frac{\gamma \vdash e : \tau}{\gamma \vdash x := e : \tau'} \quad \gamma(x) = \tau, \tau' \sqsubseteq \tau$$

Example

- Let $\gamma(x) = L$ and $\gamma(y) = H$

$$\frac{\frac{\overline{\gamma \vdash x : L} \quad \overline{\gamma \vdash 1000 : L}}{\gamma \vdash x > 1000 : L} \quad \frac{\overline{\gamma \vdash 5 : L}}{\gamma \vdash x := 5 : L} \quad \frac{\text{TODO}}{\gamma \vdash y := y + x * 3 : L}}{\gamma \vdash \text{if } x > 1000 \text{ then } x := 5 \text{ else } y := y + x * 3 : L}$$

Constraints: $\tau_1 \sqsubseteq \tau_2 \wedge \tau_2 \sqsubseteq L = \gamma(x)$. Thus $\tau_1 = \tau_2 = L$.

Fitting Rule

$$\text{(ASSIGN)} \quad \frac{\gamma \vdash e : \tau}{\gamma \vdash x := e : \tau'} \quad \gamma(x) = \tau, \tau' \sqsubseteq \tau$$

Example

- Let $\gamma(x) = L$ and $\gamma(y) = H$

$$\frac{\frac{\dots}{\gamma \vdash x > 1000 : L} \quad \frac{\dots}{\gamma \vdash x := 5 : L} \quad \frac{TODO}{\gamma \vdash y := y + x * 3 : L}}{\gamma \vdash \text{if } x > 1000 \text{ then } x := 5 \text{ else } y := y + x * 3 : L}$$

Constraints:

Fitting Rule

$$\text{(ASSIGN)} \quad \frac{\gamma \vdash e : \tau}{\gamma \vdash x := e : \tau'} \quad \gamma(x) = \tau, \tau' \sqsubseteq \tau$$

Example

- Let $\gamma(x) = L$ and $\gamma(y) = H$

$$\frac{\frac{\dots}{\gamma \vdash x > 1000 : L} \quad \frac{\dots}{\gamma \vdash x := 5 : L} \quad \frac{\frac{\frac{\gamma \vdash y : H}{\gamma \vdash x * 3 : H} \quad \frac{\gamma \vdash x : H \quad \gamma \vdash 3 : H}{\gamma \vdash x * 3 : H}}{\gamma \vdash y + x * 3 : H}}{\gamma \vdash y := y + x * 3 : L}}{\gamma \vdash \text{if } x > 1000 \text{ then } x := 5 \text{ else } y := y + x * 3 : L}$$

Constraints: $\gamma(y) \sqsupseteq L$

Fitting Rule

(VAR) $\frac{}{\gamma \vdash x : \tau} \gamma(x) \sqsubseteq \tau$

It types!

Semantics

- To make a **meaningful** statement about the language, we must define what programs **mean**.
- Since we are reading and writing variables in our program, we need a concept of memory μ :
 - ★ $\mu(x)$ means reading the memory location for variable x (yields an integer if x is defined)
 - ★ $\mu[x := n]$ means writing the integer n into the memory location of variable x
- We define the semantics as two relations:
 - ★ For expressions e we define

$$\mu \vdash e \Rightarrow n$$

to mean: at memory state μ , the expression e gets evaluated to value n .

- ★ For commands c we define

$$\mu \vdash c \Rightarrow \mu'$$

to mean: starting at memory state μ , the execution of command c changes the memory state to μ' .

Semantics

Definition (Semantics of Expressions)

$$\text{(BASE)} \quad \overline{\mu \vdash n \Rightarrow n}$$

$$\text{(CONTENT)} \quad \overline{\mu \vdash x \Rightarrow \mu(x)}$$

$$\text{(ADD)} \quad \frac{\mu \vdash e \Rightarrow n \quad \mu \vdash e' \Rightarrow n'}{\mu \vdash e + e' \Rightarrow n + n'}$$

Similar for other operators

Example

$$\underbrace{\begin{array}{|l} x := 3 \\ y := -1 \end{array}}_{\mu} \vdash \underbrace{3 * x + y}_e \Rightarrow 8$$

Semantics

Definition (Semantics of Commands)

$$\text{(UPDATE)} \quad \frac{\mu \vdash e \Rightarrow n}{\mu \vdash x := e \Rightarrow \mu[x := n]}$$

$$\text{(SEQUENCE)} \quad \frac{\mu \vdash c_1 \Rightarrow \mu' \quad \mu' \vdash c_2 \Rightarrow \mu''}{\mu \vdash c_1; c_2 \Rightarrow \mu''}$$

$$\text{(IFTRUE)} \quad \frac{\mu \vdash e \Rightarrow 1 \quad \mu \vdash c_1 \Rightarrow \mu'}{\mu \vdash \text{if } e \text{ then } c_1 \text{ else } c_2 \Rightarrow \mu'}$$

$$\text{(IFFALSE)} \quad \frac{\mu \vdash e \Rightarrow 0 \quad \mu \vdash c_2 \Rightarrow \mu'}{\mu \vdash \text{if } e \text{ then } c_1 \text{ else } c_2 \Rightarrow \mu'}$$

Semantics

Definition (Semantics of Commands)

$$\text{(WHILETRUE)} \quad \frac{\mu \vdash e \Rightarrow 1 \quad \mu \vdash c; \text{while } e \text{ do } c \Rightarrow \mu'}{\mu \vdash \text{while } e \text{ do } c \Rightarrow \mu'}$$

$$\text{(WHILEFALSE)} \quad \frac{\mu \vdash e \Rightarrow 0}{\mu \vdash \text{while } e \text{ do } c \Rightarrow \mu}$$

Example

$$\underbrace{\begin{array}{|l} x := 2 \\ y := 3 \end{array}}_{\mu} \vdash \underbrace{\begin{array}{l} z := 1; \\ \text{while } y > 0 \text{ do} \\ \quad z := z * x; \quad y := y - 1 \end{array}}_c \Rightarrow \underbrace{\begin{array}{|l} x := 2 \\ y := 0 \\ z := 8 \end{array}}_{\mu'}$$

Result

Theorem (Non-Interference instantiated for $L \sqsubseteq H$ -security)

- Suppose a program c satisfies information flow policy γ :
$$\gamma \vdash c : \tau \quad (\text{for any type } \tau, \text{ does not matter})$$
- Suppose μ_1 and μ_2 are memories that are equal on all *low* variables:

$$\mu_1(x) = \mu_2(x) \text{ for every } x \text{ with } \gamma(x) = L$$

- If we run the program on these memories:

$$\star \mu_1 \vdash c \Rightarrow \mu'_1$$

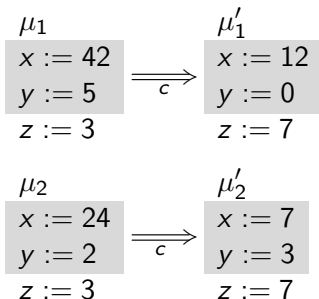
$$\star \mu_2 \vdash c \Rightarrow \mu'_2$$

(and they both terminate)

- ... then the result will be the same on all *low* variables:

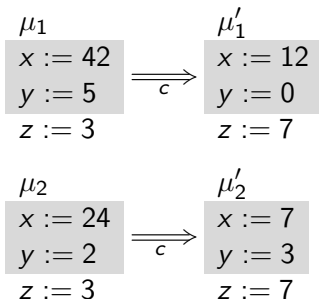
$$\mu'_1(x) = \mu'_2(x) \text{ for every } x \text{ with } \gamma(x) = L$$

Illustration



- If the low-variables (in the example: z) of μ_1 and μ_2 agree,
- then they still agree after running any command c that satisfies information flow.

Illustration



- If the low-variables (in the example: z) of μ_1 and μ_2 agree,
- then they still agree after running any command c that satisfies information flow.
- Thus, from observing the low variables we get no information about the high variables.

Relation to Secrecy and Privacy

- Note that we do **not assume** that the values in the high variables are **strong secrets**. They may contain...
 - ★ personal information like name, age, CPR, medical data...
 - ★ a guessable password
 - ★ a credit card number and expiration information
 - ★ a vote

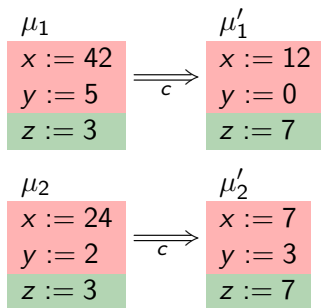
Relation to Secrecy and Privacy

- Note that we do **not assume** that the values in the high variables are **strong secrets**. They may contain...
 - ★ personal information like name, age, CPR, medical data...
 - ★ a guessable password
 - ★ a credit card number and expiration information
 - ★ a vote
- Information Flow/Noninterference guarantees that the intruder does not learn anything about it as long as he can only read low variables.

Relation to Secrecy and Privacy

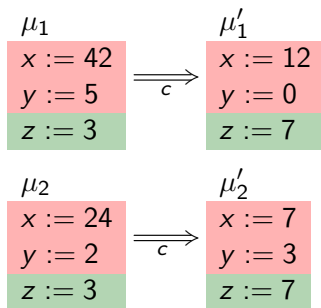
- Note that we do **not assume** that the values in the high variables are **strong secrets**. They may contain...
 - ★ personal information like name, age, CPR, medical data...
 - ★ a guessable password
 - ★ a credit card number and expiration information
 - ★ a vote
- Information Flow/Noninterference guarantees that the intruder does not learn anything about it as long as he can only read low variables.
- It is thus...
 - ★ ... not like secrecy in OFMC
 - ★ ... a bit like guessable secrecy in OFMC
 - ★ ... close to α - β -privacy!

Noninterference for Authentication / Integrity



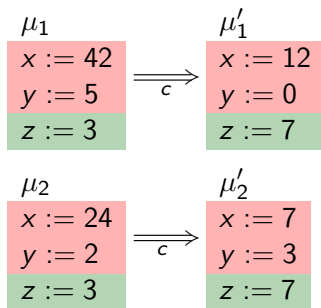
- Suppose we define the variables x and y as type **Untrusted** and z as type **Trusted**.
- Suppose **Trusted** $\sqsubseteq$ **Untrusted**.

Noninterference for Authentication / Integrity



- Suppose we define the variables x and y as type **Untrusted** and z as type **Trusted**.
- Suppose **Trusted** $\sqsubseteq$ **Untrusted**.
- Suppose further the intruder can manipulate the untrusted variables, but not the trusted ones.

Noninterference for Authentication / Integrity



- Suppose we define the variables x and y as type **Untrusted** and z as type **Trusted**.
- Suppose **Trusted** $\sqsubseteq$ **Untrusted**.
- Suppose further the intruder can manipulate the untrusted variables, but not the trusted ones.
- Then $\gamma \vdash c : \tau$ guarantees us that the intruder cannot do anything that has any influence on the trusted variables.

Result

Theorem (Non-Interference instantiated for $T \subseteq U$)

- Suppose a program c satisfies information flow policy γ :
$$\gamma \vdash c : \tau \quad (\text{for any type } \tau, \text{ does not matter})$$
- Suppose μ_1 and μ_2 are memories that are equal on all *trusted* variables:

$$\mu_1(x) = \mu_2(x) \text{ for every } x \text{ with } \gamma(x) = T$$

- If we run the program on these memories:

$$\star \mu_1 \vdash c \Rightarrow \mu'_1$$

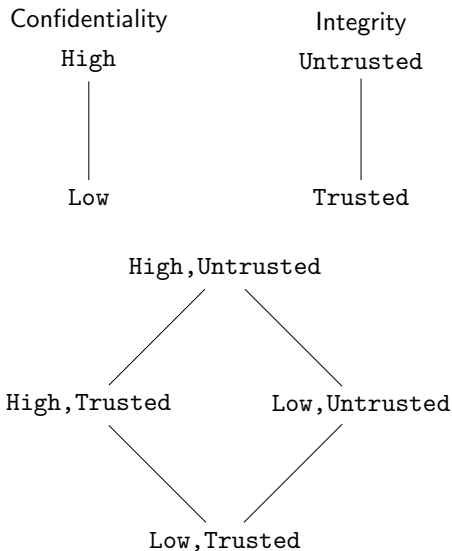
$$\star \mu_2 \vdash c \Rightarrow \mu'_2$$

(and they both terminate)

- ... then the result will be the same on all *trusted* variables:

$$\mu'_1(x) = \mu'_2(x) \text{ for every } x \text{ with } \gamma(x) = T$$

Confidentiality and Integrity in One Go?



Question

How can we become more general?

Answer

Security Policy Frameworks
for Mandatory Access Control.

Security Policy Framework

A **security policy framework** is a 4-tuple $(S, \sqsubseteq, \sqcup, \sqcap)$ where

- S is a **finite** and **non-empty** set of **security labels**.

Security Policy Framework

A **security policy framework** is a 4-tuple $(S, \sqsubseteq, \sqcup, \sqcap)$ where

- S is a **finite** and **non-empty** set of **security labels**.
- $\sqsubseteq$: $S \times S$ is a **binary relation**
 - ★ (a) $\sqsubseteq$ is **reflexive** : for all $s \in S$: $s \sqsubseteq s$
 - ★ (b) $\sqsubseteq$ is **transitive**: for all $s_1, s_2, s_3 \in S$:
$$s_1 \sqsubseteq s_2 \wedge s_2 \sqsubseteq s_3 \Rightarrow s_1 \sqsubseteq s_3$$
 - ★ (c) $\sqsubseteq$ is **anti-symmetric**: for all $s_1, s_2 \in S$:
$$s_1 \sqsubseteq s_2 \wedge s_1 \sqsupseteq s_2 \implies s_1 = s_2$$

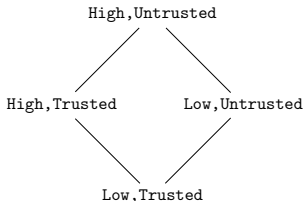
Security Policy Framework

A **security policy framework** is a 4-tuple $(S, \sqsubseteq, \sqcup, \sqcap)$ where

- S is a **finite** and **non-empty** set of **security labels**.
- $\sqsubseteq$: $S \times S$ is a **binary relation**
 - ★ (a) $\sqsubseteq$ is **reflexive** : for all $s \in S$: $s \sqsubseteq s$
 - ★ (b) $\sqsubseteq$ is **transitive**: for all $s_1, s_2, s_3 \in S$:
$$s_1 \sqsubseteq s_2 \wedge s_2 \sqsubseteq s_3 \Rightarrow s_1 \sqsubseteq s_3$$
 - ★ (c) $\sqsubseteq$ is **anti-symmetric**: for all $s_1, s_2 \in S$:
$$s_1 \sqsubseteq s_2 \wedge s_1 \sqsupseteq s_2 \implies s_1 = s_2$$
- $\sqcup$: $S \times S \rightarrow S$ and $\sqcap$: $S \times S \rightarrow S$ are two operations for **combining labels** such that
for all $s_1, s_2 \in S$:
$$s_1 \sqsubseteq s_1 \sqcup s_2 \text{ and } s_2 \sqsubseteq s_1 \sqcup s_2$$
$$s_1 \sqcap s_2 \sqsubseteq s_1 \text{ and } s_1 \sqcap s_2 \sqsubseteq s_2$$

Security Lattice

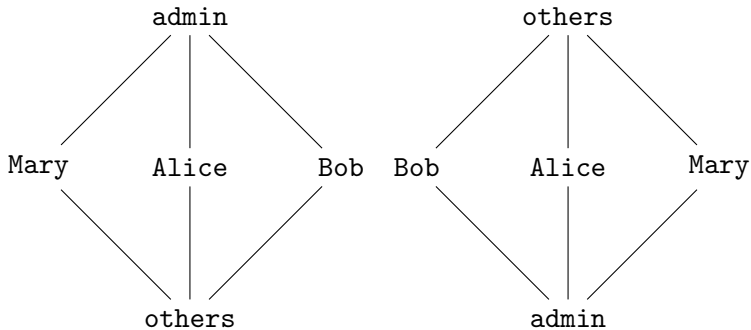
$(S, \sqsubseteq)$ is a *lattice*. This allows to depict it nicely by just denoting the direct successors of security classes:



In a security framework $(S, \sqsubseteq, \sqcup, \sqcap)$

- there is the bottom security label $\perp$ that can be obtained as $s_1 \sqcap s_2 \dots \sqcap s_n$ (if $S = \{s_1, \dots, s_n\}$).
- there is the top security label $\top$ that can be obtained as $s_1 \sqcup s_2 \dots \sqcup s_n$ (if $S = \{s_1, \dots, s_n\}$).

Example



The set of security labels

$$S = \{\text{others}, \text{Mary}, \text{Alice}, \text{Bob}, \text{admin}\}$$

What about $\sqsubseteq$, $\sqcup$ and $\sqcap$?

Typical Security Policy Frameworks

Components

Start with

- A finite and non-empty set C of **security categories**.

Typical Security Policy Frameworks

Components

Start with

- A finite and non-empty set C of **security categories**.

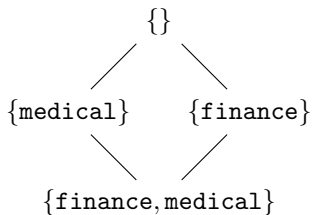
Security labels can be all subsets of C :

- $(\text{PowerSet}(C), \supseteq, \cap, \cup)$ for **confidentiality policies**.
- $(\text{PowerSet}(C), \subseteq, \cup, \cap)$ for **integrity policies**.

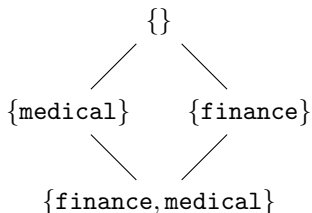
A security label $s \in \text{PowerSet}(C)$ is called a **component**.

Recall $\text{PowerSet}(C) = \{C_0 \mid C_0 \subseteq C\}$, i.e., the set of subsets of C .

Example (Confidentiality): $(\text{PowerSet}(C), \supseteq, \cap, \cup)$

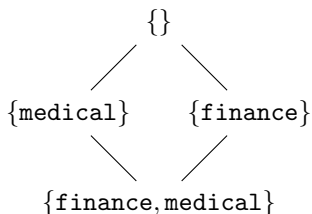


Example (Confidentiality): $(\text{PowerSet}(C), \supseteq, \cap, \cup)$



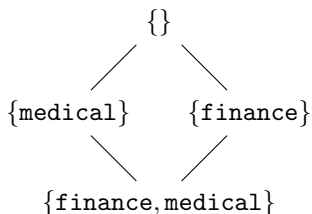
- $C = \{\text{finance}, \text{medical}\}$
- The **more departments** can **read** to the data, the **less sensitive** it is.

Example (Confidentiality): $(\text{PowerSet}(C), \supseteq, \cap, \cup)$



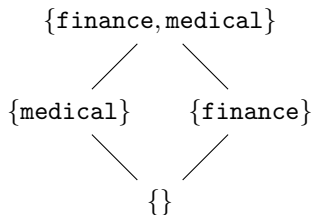
- $C = \{\text{finance}, \text{medical}\}$
- The **more departments** can **read** to the data, the **less sensitive** it is.
- Ordering labels examples ($\sqsubseteq = \supseteq$):
 $\{\text{finance}, \text{medical}\} \sqsubseteq \{\text{medical}\}$, $\{\text{medical}\} \not\sqsubseteq \{\text{finance}\}$,
 $\{\text{finance}\} \not\sqsubseteq \{\text{medical}\}$

Example (Confidentiality): $(\text{PowerSet}(C), \supseteq, \cap, \cup)$

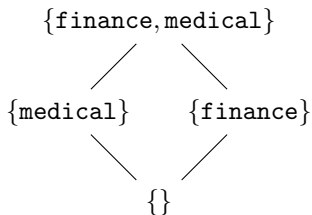


- $C = \{\text{finance, medical}\}$
- The **more departments** can **read** to the data, the **less sensitive** it is.
- Ordering labels examples ($\sqsubseteq = \supseteq$):
 $\{\text{finance, medical}\} \sqsubseteq \{\text{medical}\}$, $\{\text{medical}\} \not\sqsubseteq \{\text{finance}\}$,
 $\{\text{finance}\} \not\sqsubseteq \{\text{medical}\}$
- Combining labels examples ($\sqcup = \cap$, $\sqcap = \cup$):
 $\{\text{finance, medical}\} \sqcup \{\text{medical}\} = \{\text{medical}\}$
 $\{\text{medical}\} \sqcap \{\text{finance}\} = \{\text{finance, medical}\}$

Example (Integrity): $(\text{PowerSet}(C), \subseteq, \cup, \cap)$

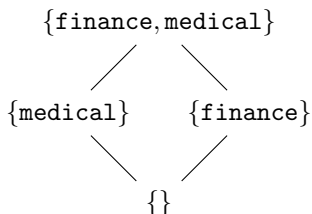


Example (Integrity): $(\text{PowerSet}(C), \subseteq, \cup, \cap)$



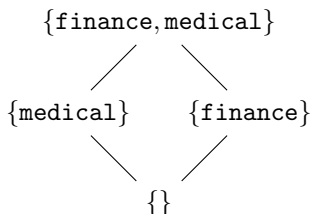
- $C = \{\text{finance}, \text{medical}\}$
- The **more departments** can **write** to the data, the **less trustworthy** it is.

Example (Integrity): $(\text{PowerSet}(C), \subseteq, \cup, \cap)$



- $C = \{\text{finance}, \text{medical}\}$
- The **more departments** can **write** to the data, the **less trustworthy** it is.
- Ordering labels examples ($\sqsubseteq = \subseteq$):
 $\{\text{medical}\} \sqsubseteq \{\text{finance}, \text{medical}\}$, $\{\text{medical}\} \not\sqsubseteq \{\text{finance}\}$,
 $\{\text{finance}\} \not\sqsubseteq \{\text{medical}\}$

Example (Integrity): $(\text{PowerSet}(C), \subseteq, \cup, \cap)$



- $C = \{\text{finance}, \text{medical}\}$
- The **more departments** can **write** to the data, the **less trustworthy** it is.
- Ordering labels examples ($\sqsubseteq = \subseteq$):
 $\{\text{medical}\} \sqsubseteq \{\text{finance}, \text{medical}\}$, $\{\text{medical}\} \not\sqsubseteq \{\text{finance}\}$,
 $\{\text{finance}\} \not\sqsubseteq \{\text{medical}\}$
- Combining labels examples ($\sqcup = \cup$, $\sqcap = \cap$):
 $\{\text{finance}\} \sqcup \{\text{medical}\} = \{\text{finance}, \text{medical}\}$
 $\{\text{medical}\} \sqcap \{\text{finance}\} = \{\}$

Combining Security Policy Frameworks

Product Construction

Whenever $(S_1, \sqsubseteq_1, \sqcup_1, \sqcap_1)$ and $(S_2, \sqsubseteq_2, \sqcup_2, \sqcap_2)$ are two security frameworks, we can construct the framework $(S, \sqsubseteq, \sqcup, \sqcap)$ where

- $S = S_1 \times S_2$
- for $(s_{11}, s_{12}), (s_{21}, s_{22}) \in S$:

$$(s_{11}, s_{12}) \sqsubseteq (s_{21}, s_{22}) \text{ iff } s_{11} \sqsubseteq_1 s_{21} \wedge s_{12} \sqsubseteq_2 s_{22}$$

- for $(s_{11}, s_{12}), (s_{21}, s_{22}) \in S$:

$$(s_{11}, s_{12}) \sqcup (s_{21}, s_{22}) = (s_{11} \sqcup_1 s_{21}, s_{12} \sqcup_2 s_{22})$$

and

$$(s_{11}, s_{12}) \sqcap (s_{21}, s_{22}) = (s_{11} \sqcap_1 s_{21}, s_{12} \sqcap_2 s_{22})$$

Example (Combining Integrity and Confidentiality):

Confidentiality

Integrity

High

Untrusted

Low

Trusted

High, Untrusted

High, Trusted

Low, Untrusted

Low, Trusted

Result

Theorem (Non-Interference for an arbitrary Security Policy Framework)

- Suppose a program c satisfies information flow policy γ :

$$\gamma \vdash c : \tau \quad (\text{for any type } \tau, \text{ does not matter})$$

- Suppose μ_1 and μ_2 are memories that are equal on all variables up to level τ_0 :

$$\mu_1(x) = \mu_2(x) \text{ for every } x \text{ with } \gamma(x) \sqsubseteq \tau_0$$

- If we run the program on these memories:

$$\star \mu_1 \vdash c \Rightarrow \mu'_1$$

$$\star \mu_2 \vdash c \Rightarrow \mu'_2$$

(and they both terminate)

- ... then the result will be the same on all variables up to level τ_0 :

$$\mu'_1(x) = \mu'_2(x) \text{ for every } x \text{ with } \gamma(x) \sqsubseteq \tau_0$$

Challenge

Let $\gamma = [x \mapsto \text{Low}, y \mapsto \text{High}]$ and the context $\text{ctx} = \text{Low}$

- Is the program $S = (\text{while } y > 0 \text{ do } y := y + 1) ; x := 2$ secure?

Use Volpano's typing rules to determine if S is secure

- Do you agree with the result of the type system?

References I



D. E. Denning and P. J. Denning.

Certification of programs for secure information flow.

Commun. ACM, 20(7):504–513, 1977.



D. M. Volpano, G. Smith, and C. E. Irvine.

A sound type system for secure flow analysis.

Journal of Computer Security, 4(2/3):167–188, 1996.